

1. Kwestie raczej oczywiste, ale wolę zaznaczyć:

- cały kod po angielsku, włącznie z komentarzami w kodzie (odpowiedzi na pytania tekstowe mogą być po polsku)
- przestrzegamy standardu kodu PEP8 w miarę możliwości, ale konwencja z dokumentacji Scikit-learn'a są ważniejsze (nawet, jak łamią PEP8)
- w szczególności używamy pythonowych, a nie javowych nazw zmiennych, funkcji, metod i klas:

```
variable_name # can be capital letters sometimes, e.g. X_train
CONSTANT_VARIABLE_NAME # for config or something like that
method_name
function_name
ClassName
```

- używamy prawie tylko i wyłącznie keyword arguments; w szczególności gdy przekazujemy 3 lub więcej argumentów
- kiedy formatujemy napisy, to unikamy dodawania napisów, zamiast tego używamy f-strings; w szczególności tyczy się to wypisywania wartości, używamy formatowania zamiast zaokrąglania explicite:

```
print(f"Accuracy: {acc_score:.2%}")
```

2. Numpy'a, Pandasa i Matplotliba importuje się ze standardowym aliasem i używa poprzez niego:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

arr = np.zeros((2, 3))
df = pd.read_csv("dataset.csv")
```

3. Ze Scikit-learn'a importuje się konkretne metody i klasy:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import GridSearchCV
```

4. Polecam formatter kodu *black*, oficjalnie utrzymywany przez Python Software Foundation (PSF). Ma też wersję do Jupyter Notebooków. Instalacja i użycie (w terminalu):

```
pip install black[jupyter]
black .
```

5. Importy w PyTorchu (lab 3 i dalsze) - raczej absolutne jak poniżej, natomiast czasem importuje się tylko konkretne klasy/funkcje, ale rzadko:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

6. Typowa konwencja zmiennych w Scikit-learn'ie jest następująca. Gdy macie wątpliwości, szukajcie przykładu w dokumentacji.

```
# 1D vector - small letters
y = np.ones(5)

# 2D matrix - capital letters
X = np.zeros((3, 4))

# typical names for regressors and classifiers
reg = LinearRegression()
clf = LogisticRegression()

# typical names; X_valid and y_valid are also used
X_train, X_test, y_train, y_test = train_test_split(...)
```